# United International University

PROJECT PROPOSAL REPORT

# Authentication & Password Security


| Group Members | Student ID |
|---|---|
| Muhmmad Ahamudul Hasan Prianto | 0112230300 |
| Mehrin Sultana | 0112230928 |


| | |
|---|---|
| **Course** | Computer Security |
| **Section** | B |
| **Group ID** | B15 |
| **Date** | 13/11/2026 |

# 1. Introduction

Passwords remain the most common way to prove identity online, and they are attacked in two very different ways: **offline**, when an attacker steals a database of password hashes and cracks them on their own hardware, and **online**, when an attacker repeatedly guesses credentials against a live login page. A secure system must withstand both.

This project, **Authentication & Password Security**, builds a complete *offensive auditing vs. defensive hardening* lifecycle. We first attack weak authentication using industry-standard cracking tools, measure how quickly it falls, and then apply layered defenses (automatic IP banning and two-factor authentication) and test whether they hold. The project uses five core tools, introduced below.

| Offensive Tools (Attackers) | Defensive Tools (Defenders) |
|---|---|
| **John the Ripper** - CPU hash cracker | **Fail2Ban + iptables** - IP shield |
| **Hashcat** - GPU hash cracker | **pyotp + qrcode** - TOTP 2FA / MFA |
| **THC-Hydra** - online brute-forcer | |

*Figure 1: Offensive vs. defensive tools in this project*

# 2. The Five Core Tools

## 2.1 John the Ripper

**Category:** Offline Password Hash Cracker (CPU-optimized)

**What it is:** A versatile, open-source password cracker that detects weak passwords by hashing dictionary words and comparing the results against stolen password hashes.

**Role in this project:**

- Simulates an **offline credential breach** in which an attacker steals and extracts password hashes.
- Attacks unsalted MD5 hashes using the `wordlists/rockyou.txt` dictionary.
- Measures baseline CPU cracking speed and cracking times.

## 2.2 Hashcat

**Category:** GPU-Accelerated Password Recovery Engine

**What it is:** Known as the world's fastest password cracker, Hashcat uses the parallel processing power of modern GPUs (via NVIDIA CUDA or OpenCL) to test billions of password candidates per second.

**Role in this project:**

- Demonstrates why fast cryptographic hashes such as MD5 are unsuitable for passwords when attacked by dedicated hardware.
- Establishes precise **hashes-per-second (H/s)** throughput benchmarks.
- Proves the effectiveness of **work factors** by comparing raw MD5 (`-m 0`) with the drastic slowdown of salted slow algorithms like bcrypt (`-m 3200`).

## 2.3 THC-Hydra

**Category:** Online Network Login Brute-Forcer

**What it is:** A fast, multithreaded network login auditing tool that runs dictionary and brute-force attacks across more than 50 protocols, including HTTP POST web forms, SSH, FTP and Telnet.

**Role in this project:**

- Simulates a live **online brute-force / credential-stuffing attack** over the network (`http-post-form`).
- Attacks the live Flask login endpoint (`/login`) using dictionary wordlists.

- Acts as the test benchmark to prove whether rate-limiting, account lockout and Fail2Ban stop live automated guessing.

### 2.4 Fail2Ban (with iptables)

**Category:** Automated Host-Based Intrusion Prevention System (IPS)

**What it is:** A Linux background daemon that scans log files for repeated failed authentication attempts and dynamically adds firewall rules (`iptables`) to block the offending IP address.

**Role in this project:**

- Provides **network-layer defense** against online brute-force attacks such as Hydra.
- Bans abusive client IPs after 5 failed attempts (`maxretry = 5`) for 15 minutes (`bantime = 900`).
- Protects server CPU and memory by dropping malicious packets at the kernel firewall before they reach the Flask application.

### 2.5 TOTP Library (pyotp + qrcode)

**Category:** Two-Factor Authentication (2FA / MFA) Engine

**What it is:** A Python implementation of **RFC 6238 (Time-based One-Time Password Algorithm)**. It generates shared secrets and computes rolling 6-digit codes that change every 30 seconds based on Unix epoch time.

**Role in this project:**

- Provides **application-layer multi-factor defense**.
- Generates QR codes at registration so authenticator apps (Google Authenticator, Authy, FreeOTP) can enroll.
- Prevents unauthorized access even if an attacker cracks or guesses the user's plaintext password.

## 3. How the Tools Work Together

Each attack tool is paired with a defense that answers it. John the Ripper and Hashcat show that fast, unsalted hashes fall quickly offline. THC-Hydra shows that an exposed login form can be hammered over the network, which motivates **Fail2Ban** and rate-limiting. Finally, **TOTP 2FA** limits the damage if a password is still compromised. Together they cover both offline database breaches and live network attacks.

## 4. Project Files & Resources

Environment Setup Recording of my project: **Drive**